

# **Project Proposal: An Intelligent but Offline Smart Doorbell**

**Team Members: Parinitha, Pranav, Anish**

## ***1. Motivation***

Most smart doorbells on the market (Ring, Nest, etc.) depend on cloud services to run face recognition and send alerts. This creates three problems: privacy risk (video/faces sent to third party servers), reliability issues (no internet means no doorbell), and recurring subscription costs. We want to explore whether a fully offline edge-deployed system can deliver similar core functionality, detecting a visitor and recognizing whether they are a known household member, using only on-device computers. This is a good fit for our team's interest in embedded systems, edge ML deployment, and resource-constrained inference, and gives us hands-on experience with the full pipeline from model training to real-time inference on limited hardware.

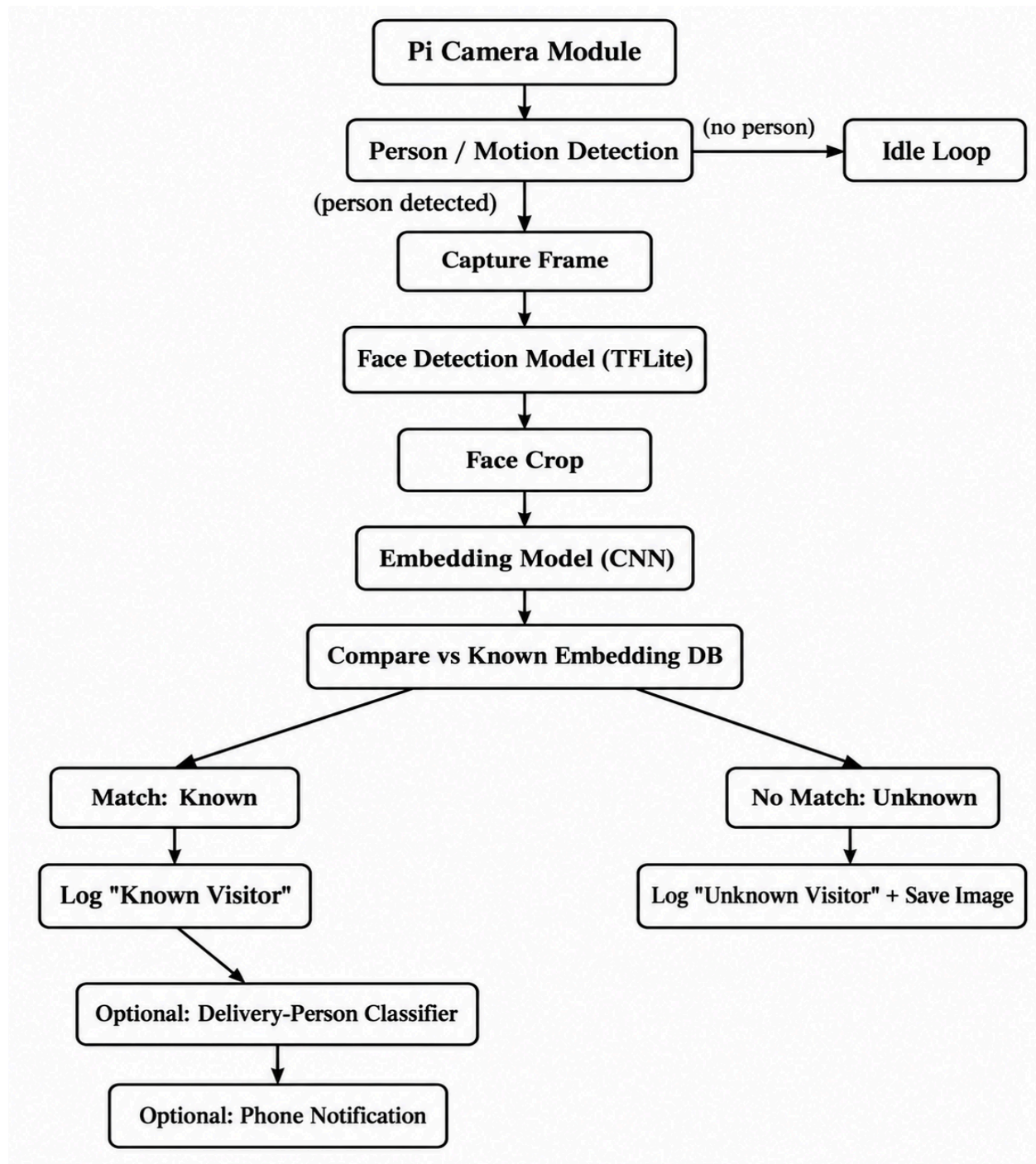
## ***2. Design Goals***

- Fully offline operation: no dependency on cloud APIs or internet connectivity at inference time.
- Low-latency inference: recognize a visitor within a few seconds of them approaching the door.
- Lightweight footprint: model(s) small enough to run in real time on a Raspberry Pi (or BeagleBone) without additional accelerators.
- Reasonable accuracy: correctly distinguish "known" vs. "unknown" visitors under normal lighting/doorway conditions.
- Extensibility: architecture should allow adding the delivery-person detection feature without a redesign.

## ***3. Deliverables***

1. A working pipeline that:
  - Captures an image via the Pi camera module when motion/a person is detected at the door.
  - Runs a lightweight person/face detection model (TensorFlow Lite) on the captured frame.
  - Computes a face embedding and compares it against a stored database of known-person embeddings to classify the visitor as known or unknown.
2. A simple alert mechanism: local log file entry (timestamp, classification, saved image) with logging kept as the primary mechanism; a push/phone notification as a stretch goal.
3. Stretch goal: a simple delivery-person classifier (e.g., detecting a person holding a box/pizza bag) as a coarse binary classifier on top of the person-detection output.
4. A short demo video/live demonstration showing end-to-end inference running directly on the edge device (not on a laptop).
5. Final code repository + README documenting setup and how to reproduce results.

#### 4. System Block Diagram (description)



Key blocks:

- Capture module: camera interface, triggers on motion or periodic polling.
- Detection module: lightweight face/person detector (e.g., MobileNet-based, per MobileNets paper).
- Recognition module: embedding extraction + nearest-neighbor match against a small local database.
- Decision/alert module: thresholding logic, logging, optional notification.
- Storage: local flat-file or SQLite database of known-person embeddings.

## 5. Hardware/Software Requirements

Hardware:

- Raspberry Pi (4B or 5 recommended for headroom) or BeagleBone
- Pi Camera Module (or USB webcam as fallback)
- MicroSD card (32GB+), power supply, optional PIR motion sensor to trigger capture

Software:

- Raspberry Pi OS (Linux)
- Python 3
- TensorFlow Lite (inference runtime on-device)
- TensorFlow/Keras or PyTorch (training, done off-device on Google Colab)
- OpenCV (image capture/preprocessing)
- A pretrained lightweight face-detection model (e.g., BlazeFace or MobileNet-SSD) , fine-tuned/converted to TFLite
- A pretrained embedding model (e.g., a small FaceNet/MobileFaceNet variant) for known-visitor matching
- SQLite or simple pickle/JSON store for embeddings

## 6. Team Member Responsibilities (Lead Roles)

| <i>Role</i>      | <i>Lead</i> | <i>Responsibilities</i>                                                                                      |
|------------------|-------------|--------------------------------------------------------------------------------------------------------------|
| Setup            | Pranav      | Pi OS setup, camera module integration, environment/dependency setup, motion-trigger wiring                  |
| Software         | Parinitha   | Core inference pipeline (capture → detect → embed → match), logging system                                   |
| Networking       | Anish       | Optional notification mechanism (local network push, e.g., via a lightweight local server or phone app hook) |
| Writing          | Parinitha   | Proposal, README, final report, documentation                                                                |
| Research         | Anish       | Model selection (face detection + embedding), literature review on lightweight edge CNNs                     |
| Algorithm Design | Pranav      | Matching/threshold logic, embedding database design, delivery-person classifier (stretch)                    |

## 7. Project Timeline

| <i>Week</i> | <i>Milestone</i>                                                                               |
|-------------|------------------------------------------------------------------------------------------------|
| Week 1      | Finalize proposal, set up Pi + camera, confirm environment (OpenCV, TFLite runtime working)    |
| Week 2      | Select/download pretrained face detection + embedding models; test on Colab with sample images |
| Week 3      | Convert models to TFLite; get basic capture → detect pipeline running on-device                |
| Week 4      | Implement embedding comparison + known/unknown classification; build known-person database     |
| Week 5      | Integrate logging; test end-to-end pipeline with real doorway conditions; tune thresholds      |
| Week 6      | Stretch goals: delivery-person detection, phone notification; start demo video/report          |
| Week 7      | Final testing, debugging, polish; record demo; finish report and README                        |
| Week 8      | Submit final deliverables / live demo                                                          |

## 8. References

1. Howard, A. G., et al. "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." arXiv:1704.04861.
2. TensorFlow Lite documentation, on-device inference guide.
3. FaceNet: A Unified Embedding for Face Recognition and Clustering (Schroff et al., 2015).
4. Raspberry Pi Camera Module documentation.

Our team divided the project into six lead roles: Pranav leads setup and algorithm design, Pari leads software development and technical writing, and Anish leads networking and research/model selection.